

CEE 6680 – SYSEN 5680/6680

Term Project: Optimal Parking via Dynamic Programming

Riya Guttigoli

Contents

1	Problem Statement	2
2	Problem Formulation	3
2.1	State, Control, and Disturbance	3
2.2	State Equation	3
2.3	Cost	4
2.4	Cost-to-go Function	4
2.5	Terminal Stage	4
2.6	Backward Recursion	4
2.7	Optimal Control Law	5
2.8	Final DP Solution Algorithm	5
2.9	Interpretation	6
3	DP Solution	6
4	Approximate DP	10
4.1	One-Step Lookahead	10
4.2	Interpretation	11
4.2.1	Policy Comparison	11
4.2.2	Cost Comparison	12
4.2.3	Stop vs Continue Justification	13
4.2.4	State-Time Policy Structure	14
4.2.5	Performance Bound	15
4.3	Wrap-up	16
5	Conclusion	16

1 Problem Statement

You are about to visit a shopping center and you seek to find a parking space close to the main entrance. Every parking space has equal probability to be free regardless whether other spaces are free or not. Suppose that you can see if a parking space is free only after you reach the space. If you decide to park your car in m spaces away from the entrance, then your cost is m . If you reach the entrance of the shopping center and you have not found a space, then your cost is M . Find the optimal decision-making strategy for you to select a parking space. You should come up with your own values of the parameters of the problem. Your code should locate all parameters in a given place appropriately marked.

2 Problem Formulation

2.1 State, Control, and Disturbance

We first establish that there are N parking spaces before the entrance.

At time $t = 0, 1, \dots, N - 1$, the current space is

$$m_t = N - t,$$

where m_t is the distance in spaces from the entrance.

Let the disturbance be

$$W_t \in \{0, 1\},$$

with

$$W_t = \begin{cases} 1, & \text{if the current parking space is empty,} \\ 0, & \text{if the current parking space is occupied.} \end{cases}$$

We assume

$$P(W_t = 1) = p, \quad P(W_t = 0) = 1 - p.$$

and that the primitive random variables W_t are independent and their distribution is known. This assumption ensures that the system satisfies the Markov property since future states depend only on the current state and not on past realizations.

The state is

$$X_t = (m_t, W_t)$$

The control input is

$$U_t \in \{0, 1\},$$

where

$$U_t = \begin{cases} 1, & \text{park now,} \\ 0, & \text{continue.} \end{cases}$$

If $W_t = 0$, the current spot is occupied, so $U_t = 1$ is not feasible. This is an MDP with an observable state and a control selected from the current state.

2.2 State Equation

If you continue, you move to the next space closer to the entrance. If you park, the process stops.

So for $t = 0, 1, \dots, N - 2$,

$$X_{t+1} = \begin{cases} (m_t - 1, W_{t+1}), & U_t = 0, \\ x_T, & U_t = 1, \end{cases}$$

where x_T refers to a terminal state.

This aligns with the standard stochastic-system format:

$$X_{t+1} = f_t(X_t, U_t, W_t)$$

2.3 Cost

If you park m_t spaces away from the entrance, your cost is m_t .

If you reach the entrance without parking, your cost is M .

So, the stopping cost is smaller when the distance is shorter.

We can integrate the cost in our solution by stating:

- if you park at time t , cost = m_t
- if you never park, terminal cost = M .

So this is inherently a stopping problem.

2.4 Cost-to-go Function

Let

$$V_t(m, w)$$

refer to the optimal expected cost from time t onward when the current space is m spaces away from the entrance and its availability is $w \in (0, 1)$.

The cost-to-go is computed backward in time from the terminal stage.

2.5 Terminal Stage

At the last space, $m = 1$:

Case 1: last space is occupied If $w = 0$, you cannot park, so you must continue to the entrance. So,

$$V_{N-1}(1, 0) = M$$

Case 2: the last space is empty If $w = 1$, you can either:

- park now and pay 1, or
- continue and pay M

So,

$$V_{N-1}(1, 1) = \min\{1, M\}.$$

Usually $M > 1$, so

$$V_{N-1}(1, 1) = 1.$$

2.6 Backward Recursion

Now suppose $m \geq 2$.

Case 1: current space is occupied

If $w = 0$, you have no decision to make and you must continue. Therefore,

$$V_t(m, 0) = E[V_{t+1}(m-1, W_{t+1})].$$

Since

$$P(W_{t+1} = 1) = p, \quad P(W_{t+1} = 0) = 1 - p,$$

this becomes

$$V_t(m, 0) = p V_{t+1}(m - 1, 1) + (1 - p) V_{t+1}(m - 1, 0).$$

Case 2: current space is empty

If $w = 1$, you compare:

- **park now:** cost = m ,
- **continue:** expected future cost

$$E[V_{t+1}(m - 1, W_{t+1})].$$

Thus

$$V_t(m, 1) = \min \{m, E[V_{t+1}(m - 1, W_{t+1})]\}.$$

Again substituting the Bernoulli probabilities,

$$V_t(m, 1) = \min \{m, p V_{t+1}(m - 1, 1) + (1 - p) V_{t+1}(m - 1, 0)\}.$$

So the DP recursion is:

$$V_t(m, 0) = p V_{t+1}(m - 1, 1) + (1 - p) V_{t+1}(m - 1, 0),$$

$$V_t(m, 1) = \min \{m, p V_{t+1}(m - 1, 1) + (1 - p) V_{t+1}(m - 1, 0)\}.$$

2.7 Optimal Control Law

From the recursion above, if the current spot is empty, the optimal decision is:

$$U_t^* = \begin{cases} 1, & \text{if } m \leq p V_{t+1}(m - 1, 1) + (1 - p) V_{t+1}(m - 1, 0), \\ 0, & \text{if } m > p V_{t+1}(m - 1, 1) + (1 - p) V_{t+1}(m - 1, 0). \end{cases}$$

If the current spot is occupied, then automatically

$$U_t^* = 0.$$

This tells us that **if the current spot is empty, park if the current distance cost is less than or equal to the expected cost of continuing. Otherwise continue.**

2.8 Final DP Solution Algorithm

Define

$$J_m$$

as the optimal expected cost when there are m spaces left to inspect before reaching the entrance, before you observe whether the current one is empty.

Then:

Terminal condition

$$J_0 = M$$

Recursion

When there are $m \geq 1$ spaces left:

- with probability $1 - p$, the current spot is occupied, so you continue and incur J_{m-1} ,

- with probability p , the current spot is empty, and you choose the better of:
 - park now with cost m ,
 - continue with cost J_{m-1} .

Therefore,

$$J_m = (1 - p)J_{m-1} + p \min\{m, J_{m-1}\}, \quad m = 1, 2, \dots, N.$$

So the optimal decision rule is

$$\mu^*(m) = \begin{cases} 1, & \text{if the current spot is empty and } m \leq J_{m-1}, \\ 0, & \text{otherwise.} \end{cases}$$

where:

- $\mu^*(m) = 1$ means park,
- $\mu^*(m) = 0$ means continue.

2.9 Interpretation

The DP solution says:

- the closer the empty space is to the entrance, the smaller the immediate parking cost,
- but continuing gives you a chance of finding an even better space,
- so the optimal strategy compares the current stopping cost with the expected continuation cost.

3 DP Solution

The dynamic programming solution was coded using the cost-to-go variable J_m , where J_m represents the optimal expected cost when there are m parking spaces left to inspect before reaching the entrance. The terminal condition is

$$J_0 = M,$$

because if there are no parking spaces remaining, the driver reaches the entrance without parking and incurs cost M .

For each distance $m = 1, \dots, N$, the DP recursion is

$$J_m = (1 - p)J_{m-1} + p \min\{m, J_{m-1}\}.$$

This expression comes from the two possible realizations of the disturbance W_t , which represents whether the current parking spot is empty. With probability $1 - p$, the current spot is occupied, so the driver must continue and incurs expected cost J_{m-1} . With probability p , the spot is free, so the driver chooses between parking immediately with cost m , or continuing with expected cost J_{m-1} . The decision is obtained by comparing the immediate stopping cost with the expected continuation cost.

So, the optimal control strategy is

$$\mu^*(m) = \begin{cases} 1, & \text{if } m \leq J_{m-1}, \\ 0, & \text{if } m > J_{m-1}. \end{cases}$$

where $\mu^*(m) = 1$ means park and $\mu^*(m) = 0$ means continue. If the current spot is occupied, the only feasible action is to continue. This is an optimal stopping problem since the decision at each stage is whether

to stop (park) or continue..

To assess the validity of the solution written and executed in Matlab, I chose $N = 100$, $p = 0.3$, and $M = 25$ for to reflect a moderate probability of availability and a high penalty for failing to park.

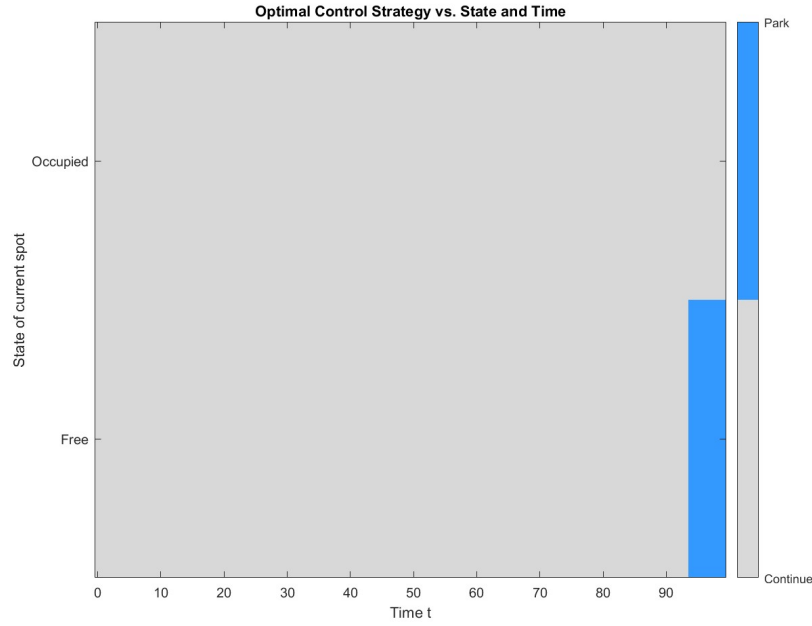


Figure 1: Optimal Control Strategy vs. State and Time

The first plot, Optimal Control Strategy vs. State and Time, shows the policy for both possible current states. When the current spot is occupied, the plot correctly shows “continue” for all times. When the spot is free, the policy changes to “park” only near the end of the search, meaning only when the driver is close enough to the entrance.

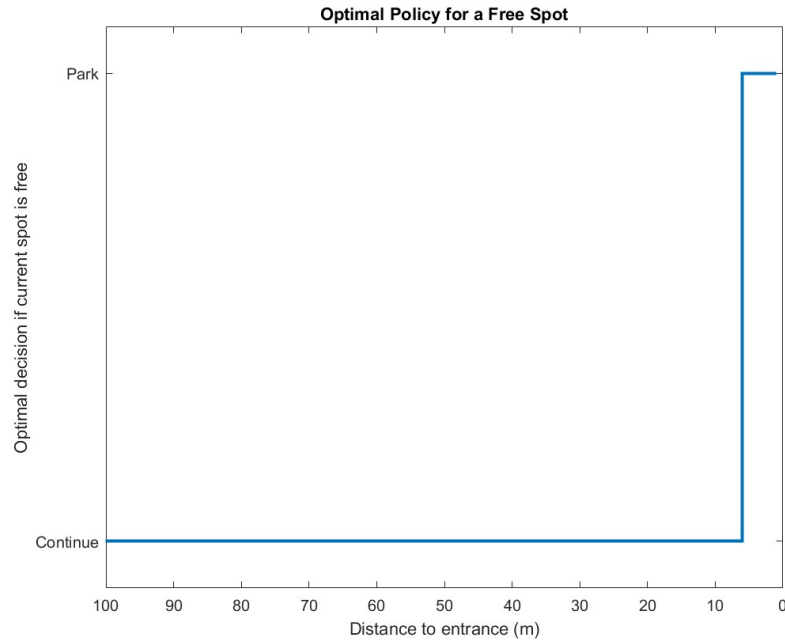


Figure 2: Optimal Policy for a Free Spot

The second plot, Optimal Policy for a Free Spot, shows the threshold structure more clearly. For large values of m , the driver continues because the parking cost is too high. For small values of m , the driver parks because the current spot is close enough to the entrance.

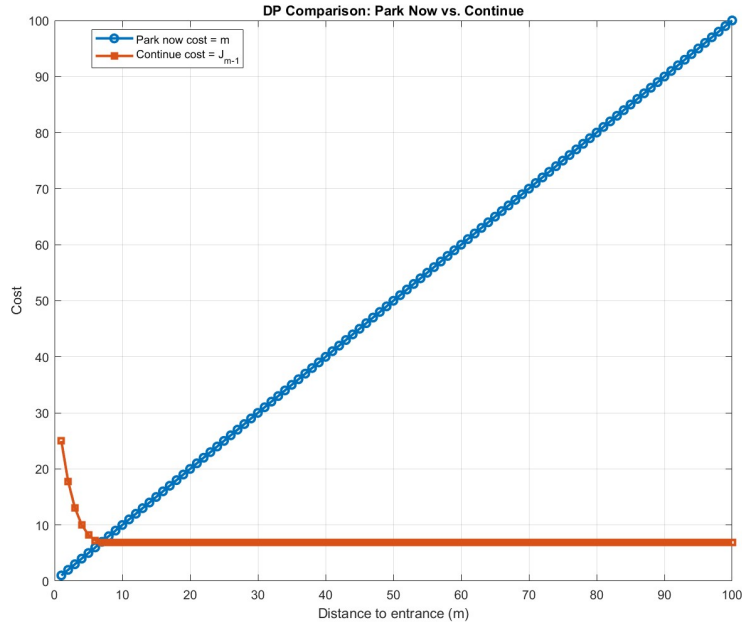


Figure 3: DP Comparison: Park Now vs. Continue

The third plot, DP Comparison: Park Now vs. Continue, directly justifies the decision rule. The blue curve is the immediate parking cost m , while the orange curve is the continuation cost J_{m-1} . The policy switches to park exactly when the parking cost becomes less than or equal to the continuation cost.

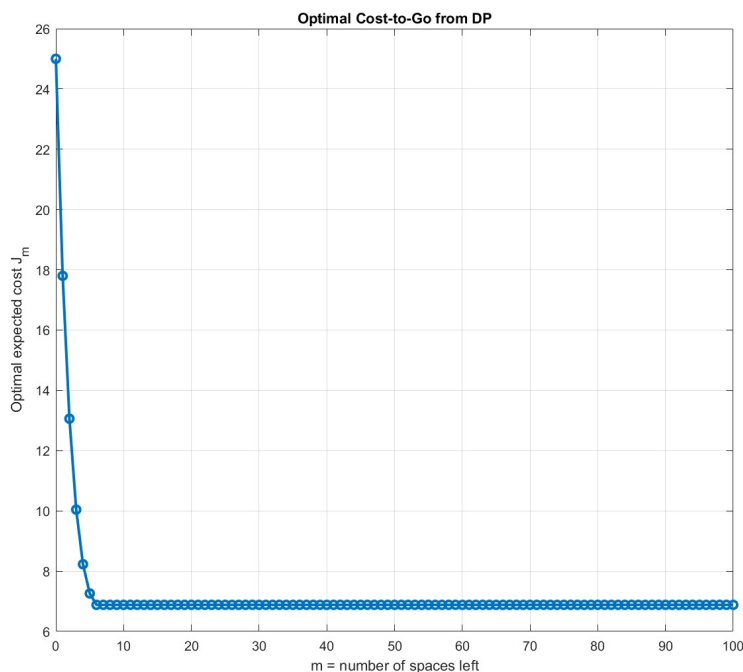


Figure 4: Optimal Cost-to-Go from DP

The fourth plot, Optimal Cost-to-Go from DP, shows that J_m decreases from $M = 25$ and eventually levels off. This means that once many spaces remain, the additional far-away spaces do not significantly improve the expected cost, because the optimal policy will ignore spots that are too far from the entrance.

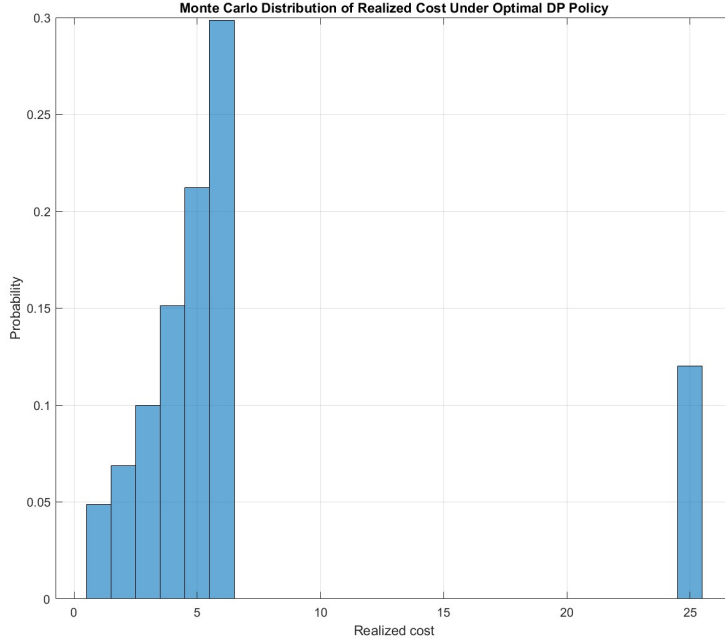


Figure 5: Monte Carlo Histogram

Finally, the Monte Carlo histogram validates the DP solution using random realizations of the parking-space availability disturbance. Each trial randomly samples whether each spot is free using $P(W_t = 1) = p$, applies the DP policy, and records the realized cost. The histogram shows most costs happening in the parking threshold region, with a separate bar at $M = 25$ for trials where no acceptable free spot is found. The Monte Carlo average should be close to the DP-predicted cost J_N , which supports that the coded DP policy is working correctly.

This optimal solution acts as a benchmark for evaluating the approximate dynamic programming approach in the next section.

4 Approximate DP

4.1 One-Step Lookahead

To reduce computational complexity, a **one-step lookahead (OSLA) approximation** is implemented. Instead of using the exact cost-to-go J_{m-1} , the continuation cost is approximated using only the immediate next step.

If the current parking spot is free and is m spaces from the entrance, the controller compares

$$\text{park now: } m$$

with the one-step lookahead continuation cost:

$$\tilde{J}_m = p(m - 1) + (1 - p)M.$$

This approximation assumes:

- If the next spot is free $\rightarrow$ park there at cost $m - 1$,

- If the next spot is occupied $\rightarrow$ incur terminal cost M .

So, the approximate policy is:

$$\tilde{\mu}(m) = \begin{cases} 1, & m \leq p(m-1) + (1-p)M, \\ 0, & \text{otherwise.} \end{cases}$$

This is a suboptimal stopping rule, since it replaces the true expected future cost with a simplified estimate. This approximation reduces computational complexity by avoiding backward recursion over the entire horizon, at the cost of suboptimal decisions.

4.2 Interpretation

The approximate DP solution uses the same parameters (N, p, M) as the exact DP in MATLAB to enable a direct comparison of performance.

4.2.1 Policy Comparison

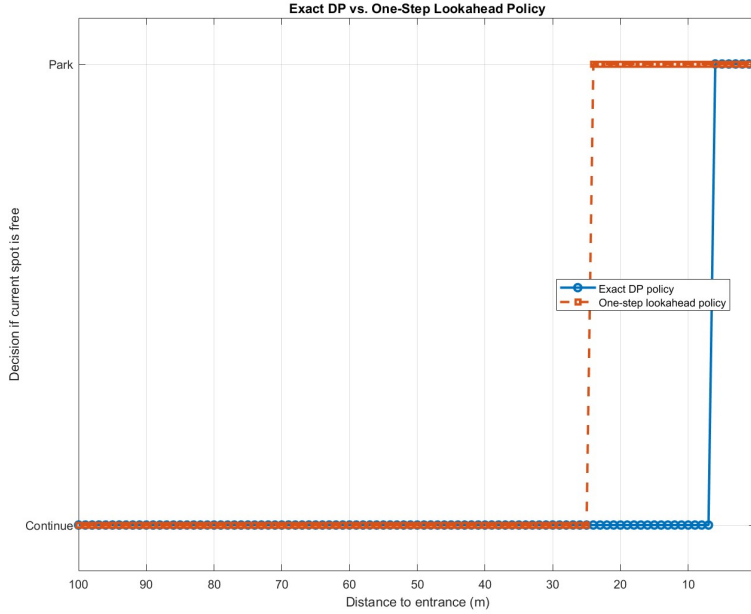


Figure 6: Exact DP vs. One-Step Lookahead Policy

The policy comparison plot shows that both the exact DP and the one-step lookahead policy exhibit a **threshold structure**, where the controller switches from “continue” to “park” at a critical distance.

However, the thresholds differ significantly:

- **Exact DP threshold:** approximately $m \approx 8$
- **One-step lookahead threshold:** approximately $m \approx 25$

This indicates that the approximate policy parks much earlier than the optimal policy.

This behavior arises because the one-step lookahead approximation overestimates the cost of continuing, making future opportunities appear worse than they actually are.

4.2.2 Cost Comparison

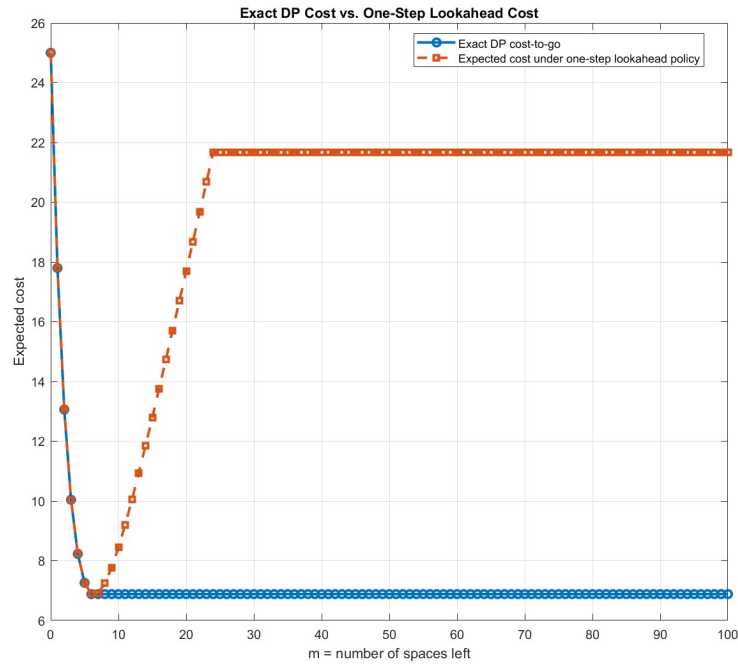


Figure 7: Exact DP Cost vs. One-Step Lookahead Cost

The plot comparing exact and approximate expected costs shows a clear gap:

- The exact DP cost quickly decreases and stabilizes at a low value (~ 7)
- The approximate cost increases and stabilizes at a much higher value (~ 21 – 22)

This demonstrates that the one-step lookahead policy is significantly suboptimal.

The reason is that the approximate method ignores the possibility of finding better parking spots beyond the next step, which is captured in the full DP recursion.

4.2.3 Stop vs Continue Justification

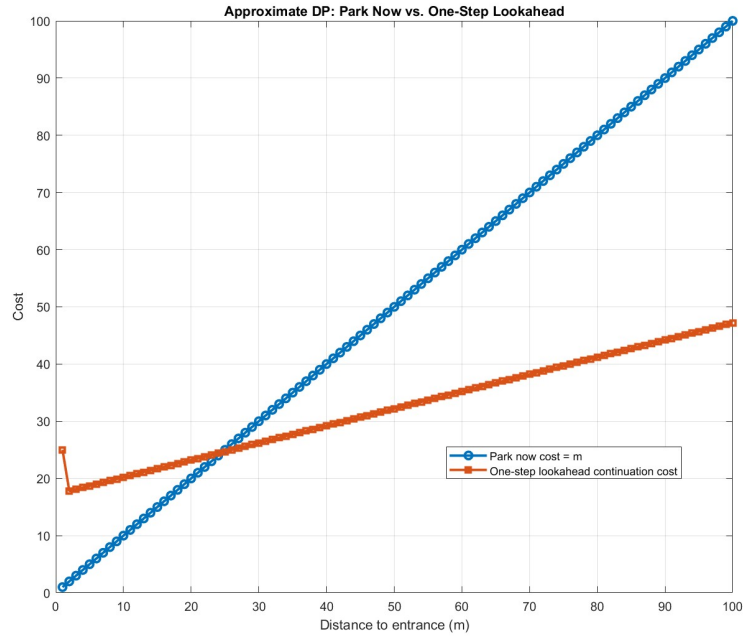


Figure 8: Approximate DP: Park Now vs One-Step Lookahead

The plot comparing:

- **Immediate cost m**
- **One-step continuation cost**

shows that the approximate policy switches decisions at the intersection point (~ 25).

This directly explains the threshold behavior in the approximate policy.

4.2.4 State-Time Policy Structure

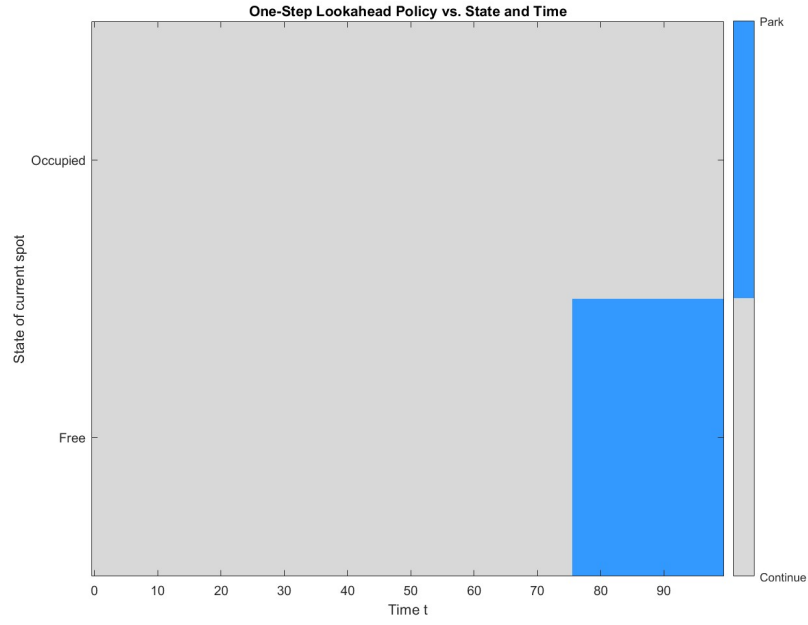


Figure 9: One-Step Lookahead Policy vs. State and Time

The state-time plot confirms that:

- If the spot is occupied $\rightarrow$ always continue
- If the spot is free $\rightarrow$ decision follows a threshold rule

This matches the structure of an optimal stopping problem and is consistent with the DP formulation.

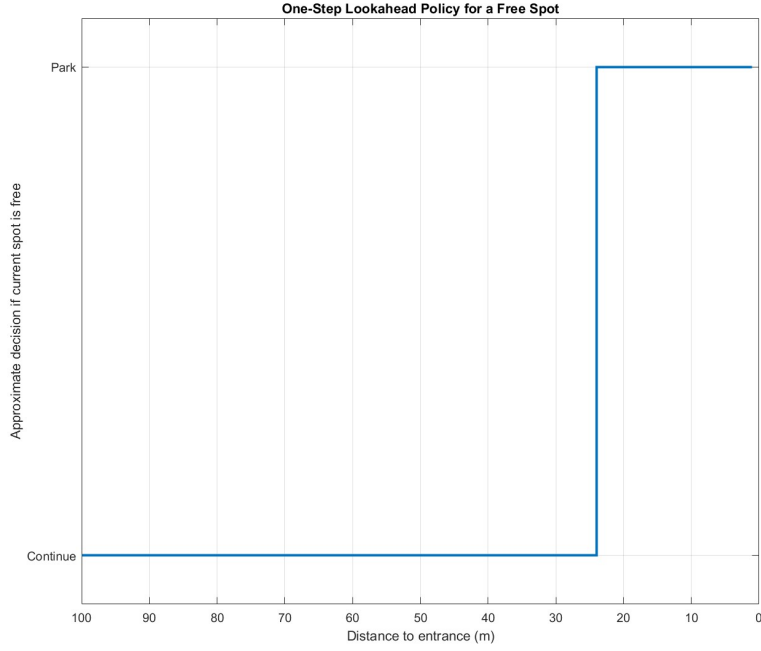


Figure 10: One-Step Lookahead Policy for a Free Spot

4.2.5 Performance Bound

Let:

- J_N = exact DP cost
- $\tilde{J}_N$ = approximate policy cost

From the results:

- Exact cost ≈ 7
- Approximate cost ≈ 21.5

So the performance gap is:

$$\tilde{J}_N - J_N \approx 14.5$$

Relative error:

$$\frac{\tilde{J}_N - J_N}{J_N} \approx 200\%$$

This confirms that:

$$\tilde{J}_N \geq J_N$$

as expected, since DP is optimal.

This large relative error emphasizes that short-horizon approximations can produce poor outcomes in stochastic optimal stopping problems where future opportunities are significant.

4.3 Wrap-up

Overall, the one-step lookahead policy maintains the qualitative structure of the optimal policy but significantly degrades performance. This highlights the importance of accurately modeling future uncertainty in stochastic decision problems, specifically in optimal stopping scenarios.

5 Conclusion

This project formulated and solved the parking problem as a finite-horizon stochastic dynamic programming problem. The exact DP solution produced an optimal threshold policy, where the decision to park or continue depends on comparing the immediate cost of stopping with the expected future cost. The results showed that incorporating full future information leads to a significantly lower expected cost and a well-defined switching point near the entrance.

To reduce computational complexity, a one-step lookahead (OSLA) approximation was introduced. While this approach simplifies the decision-making process by considering only immediate future outcomes, it was shown to produce a substantially different policy. In particular, the approximate policy stops much earlier than the optimal policy due to overestimating the cost of continuing.

Comparative analysis demonstrated a large performance gap between the exact and approximate solutions, with the OSLA policy yielding a significantly higher expected cost. This highlights the importance of accounting for future opportunities in stochastic decision problems and illustrates the tradeoff between computational simplicity and decision quality.

Overall, this study reinforces a key principle in dynamic programming: accurate modeling of future uncertainty is critical for optimal decision-making.